Basic Python Part 2 - Control Flow and Functions

In this section we will go over the basic control flow features in Python -- for-loops, if-then blocks, and functions.

Documentation: Control Flow

`for`-loops

Python has a number of ways to iterate, or "loop" over commands. The most common is the `for` loop, which allows a single block of code to be executed multiple times in sequence:

In [1]:
```
for i in range(6):
    print(i)

print("All done.")
```

```
0
1
2
3
4
5
All done.
```

The code above does the following:

1. `range(6)` creates a `range` type of 6 consecutive integers starting at 0.
2. For each integer in the list, assign the integer to the variable `i` and then execute the code inside the `for` loop. This means that the statement `print(i)` is executed 6 times, each time with a different value assigned to `i`.
3. Print "All done.'

If you're coming from almost any other programming language, the above code may look strange to you--how does Python know that `print(i)` is meant to be executed 6 times, whereas `print "All done."` should only be executed once? Most languages would require you to explicitly mark the end of `for` loop (for example, by writing `END`, `ENDFOR`, or a closing brace `}`).

The answer is that Python uses **indentation** to determine the beginning and end of code blocks. Every line with the same initial indentation is considered a part of the same code block. A different indentation level indicates another code block. This makes many newcomers uncomfortable at first! Never fear; we love this feature for its simplicity and cleanliness, and our text editors take care of the tedious work for us.

Let's see what would have happened if the `print(i)` statement were not correctly indented:

In [2]:
```
for i in range(10):
print(i)
```

```
  Cell In[2], line 2
    print(i)
    ^
IndentationError: expected an indented block after 'for' statement on line 1
```

The code in the previous cell gives an `IndentationError`, which tells us that we need to fix the indentation of the line that says `print(i)`.

How much should you indent? By convention, most Python code indents using 4 spaces. This is good practice. Technically, you can choose as much white space you would like as long as it is **consistent**, but please do not use tabs! This can confuse other people trying to use your code on a different platform or with a different editor. Most Python environments will by default or can be configured to convert the use of the tab key to 4 spaces. Many will also auto-indent what should be a new code block.

We will see many more examples of code indentation below.

Exercise 2.1: Try typing the above for-loop into the cell below. Notice that when you press `enter` at the end of the first line, Jupyter will automatically begin an indented code block.

In [4]:
```
for i in range(10):
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

Exercise 2.2: Print out the squares of the first ten integers.

In [5]:
```
for i in range(10):
    print(i*i)
```

```
0
1
4
9
16
25
36
49
64
81
```

Exercise 2.3: Print out the sum of the first ten integers.

In [6]:
```
# simple way
isum = 0
for i in range(10):
    isum += i
print(isum)

# one-liner using built-in function
print(sum(range(10)))
```

```
45
45
```

Exercise 2.4: Add two equal length lists of numbers element-wise.

In [7]:
```
X = [1,2,3,4,5]
Y = [7,6,3,3,4]

Z = []
for i in range(len(X)):
    Z.append(X[i] + Y[i])
print(Z)
```

```
[8, 8, 6, 7, 9]
```

Conditionals: if...elif...else

`if` statements allow your code to make decisions based on the value of an object.

In [8]:
```
x = 5

if x == 5:
    # this line will be executed
    print("x equals 5!")

if x < 2:
    # this line will NOT be executed
    print("x is less than two.")
```

```
x equals 5!
```

> Again, the body of the `if` code block is defined by indentation.
>
> Note the use of the double-equal sign to indicate a test for equality. Remember:
>
> - A single-equal `=` is an **assignment**; this changes the value of a variable.
> - A double-equal `==` is a **question**; it returns True if the objects on either side are equal.

Exercise 2.5:

Repeat the above code block with the if statement changed to

`if x=5:`

What do you get?

In [9]:
```
x = 5

if x = 5:
    # this line will be executed
    print("x equals 5!")
```

```
  Cell In[9], line 3
    if x = 5:
       ^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

> The expression `x==5` evaluates to a boolean value (either `True` or `False`).

In [10]:
```
print(x==5)
print(type(x==5))
```

```
True
<class 'bool'>
```

> Any type of expression that can be evaluated to a boolean may be used as a condition in an if-block. For example:

In [11]:
```
# Tests for inequality
print(x > 5)   # greater than
print(x < 5)   # less than
print(x >= 5)  # greater than or equal
print(x <= 5)  # less than or equal
print(x != 5)  # not equal
```

```
False
False
True
True
False
```

In [12]:
```
# Compound boolean tests
print((x > 2) and (x < 10))
print((x < 2) or not (x > 10))
```

```
True
True
```

In [13]:
```
# Tests for inclusion
print(7 in range(10))
print('x' not in ['a', 'b', 'c'])
```

```
True
True
```

> One can also add `elif` and `else` blocks to an `if` statement. `elif` adds another conditional to be tested in case the first is false. The `else` block is run only if all other conditions are false. `elif` and `else` are both optional.

In [14]:
```
x = 5

# There are three possible blocks for Python to execute here.
# Only the *first* block to pass its conditional test will be executed.
if x == 0:
    print("x is 0")
elif x < 0:
    print("x is negative")
else:
    print("x is positive")
```

```
x is positive
```

> ## Mixing for-loops and conditionals
>
> `for` loops also support `break` and `continue` statements. These follow the standard behavior used in many other languages:
>
> - `break` causes the currently-running loop to exit immediately--execution jumps to the first line **after** the end of the for-loop's block.
> - `continue` causes the current iteration of the for-loop to end--execution jumps back to the beginning of the for-loop's block, and the next iteration begins.

In [15]:
```
for i in range(10):
    if i == 3:
        break
    print(i)
```

```
0
1
2
```

In [16]:
```
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)
```

```
1
3
5
7
9
```

> Note the use of **nested** indentation--we have an `if` block nested inside a `for`-loop.

Exercise 2.6: Most code editors provide some means to automatically indent and de-indent whole blocks of code. In the cell above, select the last three lines and then press `tab` to increase their indentation, and `shift-tab` to decrease their indentation.

In [17]:
```
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)
```

```
1
3
5
7
9
```

Iterating over lists, tuples, and dicts

A very common operation in programming is to loop over a list, performing the same action once for each item in the list. How does this work in Python?

For example, if you want to print each item in a list, one at a time, then you might try the following:

In [18]:
```
words_from_hello = 'hello, world.  How are you?'.split()

number_of_words = len(words_from_hello)

for i in range(number_of_words):
    print(words_from_hello[i])
```

```
hello,
world.
How
are
you?
```

> Is there a better way?
>
> When we originally used `for i in range(6)`, we said that `range(6)` just generates a sequence of integers, and that the for-loop simply executes its code block once for each item in the sequence.
>
> In this case, we already **have** the list of items we want to iterate over in `words_from_hello`, so there is no need to introduce a second sequence of integers:

In [19]:
```
for word in words_from_hello:
    print(word)
```

```
hello,
world.
How
are
you?
```

> In each iteration of the loop, the next succesive element in the list `words_from_hello` is assigned to the value `word`. The variable `word` can then be used inside the loop.
>
> We can also iterate over `tuple`s and `dict`s. Iterating over a `tuple` works exactly as with `list`s:

In [20]:
```
for i in (1,2,3,4):
    print(i)
```

```
1
2
3
4
```

> If we use a `dict` in a for-loop, then the behavior is to iterate over the **keys** of the `dict`:

In [21]:
```
fruit_dict = {'apple':'red',  'orange':'orange',  'banana':'yellow'}

# inside the for-loop, fruit will be assigned to the *keys* from fruit_dict,
# not the *values*.
for fruit in fruit_dict:
    # for each key, retrieve and print the corresponding value
    print(fruit_dict[fruit])
```

```
red
orange
yellow
```

> Are the values printed in the order you expected? Be careful! `dict`s are not ordered, so you cannot rely on the order of the sequence that is returned.
>
> There are actually a few different ways to iterate over a `dict`:

In [22]:
```
print("Iterate over keys:")
# (this is the same behavior as shown above)
for key in fruit_dict.keys():
    print(key)

print("")

print("Iterate over values:")
for value in fruit_dict.values():
    print(value)

print("")

print("Iterate over keys AND values simultaneously:")
for key,value in fruit_dict.items():
    print(key, value)
```

```
Iterate over keys:
apple
orange
banana

Iterate over values:
red
orange
yellow

Iterate over keys AND values simultaneously:
apple red
orange orange
banana yellow
```

> The last example is a nice combination of **multiple assignment** and iterators. The `items()` method generates a tuple for each loop iteration that contains the `(key, value)` pair, which is then split and assigned to the `key` and `value` variables in the for-loop.

It is quite common that we will want to iterate over a list of items **and** at the same time have access to the index (position) of each item within the list. This can be done simply using the tools shown above, but Python also provides a convenient function called `enumerate()` that makes this easier:

In [23]:
```
for i,word in enumerate(words_from_hello):
    print(i, word)
```

```
0 hello,
1 world.
2 How
3 are
4 you?
```

Exercise 2.7: Create a dictionary with the following keys and values by iterating with a `for` loop. Start with an empty `dict` and add one key/value pair at a time.

In [24]:
```
key_list   = ['smuggler', 'princess','farmboy']
value_list = ['Han', 'Leia', 'Luke']

# simple way
empty = {}
for i in range(3):
    empty[key_list[i]] = value_list[i]

# using built-in zip function
empty = {}
for key, value in zip(key_list, value_list):
    empty[key] = value

# using dictionary comprehension
empty = { key:value for key,value in zip(key_list, value_list) }

print(empty)
```

```
{'smuggler': 'Han', 'princess': 'Leia', 'farmboy': 'Luke'}
```

List Comprehensions

Earlier, we said that a very common operation in programming is to loop over a list, performing the same action once for each item in the list. Python has a cool feature to simplify this process. For example, let's take a list of numbers and compute a new list containing the squares of those numbers:

In [25]:
```
# First, the long way:
numbers = [2, 6, 3, 8, 4, 7]
squares = []
for n in numbers:
```

```
    squares.append(n**2)

print(squares)
```

```
[4, 36, 9, 64, 16, 49]
```

> A **list comprehension** performs the same operation as above, but condensed into a single line.

In [26]:
```
# A one-line version of the same operation:
squares = [n**2 for n in numbers]

print(squares)
```

```
[4, 36, 9, 64, 16, 49]
```

> List comprehensions can also be used to filter lists using `if` statements. For example, the following prints out the first 100 even valued perfect squares.

In [27]:
```
# The long way:
even_squares = []
for x in range(10):
    if x%2 == 0:
        even_squares.append(x * x)

print(even_squares)

# The short way:
even_squares = [x*x for x in range(10) if x%2 == 0]

print(even_squares)
```

```
[0, 4, 16, 36, 64]
[0, 4, 16, 36, 64]
```

> List comprehensions are frequently-used by Python programmers, so although it is not necessary to use list comprehensions in your own code, you should still understand how to read them.

> ## Functions
>
> Code execution is often very repetitive; we have seen this above with for-loops. What happens when you need the same block of code to be executed from two different parts of your program? Beginning programmers are often tempted to copy-and-paste snippets of code, a practice that is highly discouraged. In the vast majority of cases, code duplication results in programs that are difficult to read, maintain, and debug.
>
> To avoid code duplication, we define blocks of re-usable code called **functions**. These are one of the most ubiquitous and powerful features across all programming languages. We have already seen a few of Python's built-in functions above-- `len()`,

`enumerate()`, etc. In this section we will see how to define new functions for our own use.

In Python, we define new functions by using the following syntax:

In [28]:
```
# Define a function called "my_add_func" that requires 2 arguments (inputs)
# called "x" and "y"
def my_add_func(x, y):
    # The indented code block defines the behavior of the function
    z = x + y
    return z

# Run the function here, with x=3 and y=5
print(my_add_func(3, 5))
```

```
8
```

The first line (1) defines a new function called `my_add_func`, and (2) specifies that the function requires two arguments (inputs) called `x` and `y`.

In the last line we actually execute the function that we have just defined. When the Python interpreter encounters this line, it immediately jumps to the first line of `my_add_func`, with the value 3 assigned to `x` and 5 assigned to `y`. The function returns the sum `x + y`, which is then passed to the `print()` function.

The indented code block in between defines what will actually happen whenever the function is run. All functions must either return an object--the output of the function--or raise an exception if an error occurred. Any type of object may be returned, including tuples for returning multiple values:

In [29]:
```
def my_new_coordinates(x, y):
    return x+y, x-y

print(my_new_coordinates(3,5))
```

```
(8, -2)
```

Another nice feature of Python is that it allows documentation to be embedded within the code.

We document functions by adding a string just below the declaration. This is called the "docstring" and generally contains information about the function. Calling "help" on a function returns the docstring.

Note that the triple-quote `'''` below allows the string to span multiple lines.

In [30]:
```
def my_new_coordinates(x, y):
    '''Returns a tuple consisting of the sum (x+y) and difference (x-y) of t
```

```
    This function is intended primarily for nefarious purposes.
    '''
    return x+y, x-y

help(my_new_coordinates)
```

```
Help on function my_new_coordinates in module __main__:

my_new_coordinates(x, y)
    Returns a tuple consisting of the sum (x+y) and difference (x-y) of the
arguments (x,y).

    This function is intended primarily for nefarious purposes.
```

> A function's arguments can be given a default value by using `argname=value` in the declared list of arguments. These are called "keyword arguments", and they must appear **after** all other arguments. When we call the function, argument values can be specified many different ways, as we will see below:

In [31]:
```
def add_to_x(x, y=1):
    """Return the sum of x and y.
    If y is not specified, then its default value is 1.
    """
    return x+y

# Many different ways we can call add_to_x():
print(add_to_x(5))
print(add_to_x(5, 3))
print(add_to_x(6, y=7))
print(add_to_x(y=8, x=4))
```

```
6
8
13
12
```

> You can also "unpack" the values inside lists and tuples to fill multiple arguments:

In [32]:
```
args = (3,5)

print(add_to_x(*args))   # equivalent to add_to_x(args[0],args[1])
```

```
8
```

> Alternatively, you can unpack a dictionary that contains the names and values to assign for multiple keyword arguments:

In [33]:
```
arg_dict = {'x':3, 'y':5}

print(add_to_x(**arg_dict))
```

```
8
```

Function arguments are passed by reference

Let's revisit a point we made earlier about variable names.

A variable in Python is a **reference** to an object. If you have multiple variables that all reference the same object, then any modifications made to the object will appear to affect all of those variables.

For example:

In [34]:
```
# Here's a simple function:
def print_backward(list_to_print):
    """Print the contents of a list in reverse order."""
    list_to_print.reverse()
    print(list_to_print)

# Define a list of numbers
my_list = [1,2,3]
print(my_list)

# Use our function to print it backward
print_backward(my_list)

# Now let's look at the original list again
print(my_list)
```

```
[1, 2, 3]
[3, 2, 1]
[3, 2, 1]
```

What happened in the code above? We printed `my_list` twice, but got a different result the second time.

This is because we have two variables-- `my_list` and `list_to_print` --that both reference the **same** list. When we reversed `list_to_print` from inside `print_backward()`, it also affected `my_list`.

If we wanted to avoid this side-effect, we would need to somehow avoid modifying the original list. If the side-effect is **intended**, however, then it should be described in the docstring:

In [35]:
```
# Solution 1: copy the list before reversing
def print_backward(list_to_print):
    """Print the contents of a list in reverse order."""
    list_copy = list_to_print[:]  # the [:] notation creates a copy of the l
    list_copy.reverse()
    print(list_copy)

# Solution 2: fix the docstring
```

```
def print_backward(list_to_print):
    """Reverse a list and print its contents."""
    list_to_print.reverse()
    print(list_to_print)
```

Variable Scope

When working with large programs, we might create so many variables that we lose track of which names have been used and which have not. To mitigate this problem, most programming languages use a concept called "scope" to confine most variable names to a smaller portion of the program.

In Python, the scope for any particular variable depends on the location where it is first defined. Most of the variables we have defined in this notebook so far are "global" variables, which means that we should be able to access them from anywhere else in the notebook. However, variables that are assigned inside a function (including the function's arguments) have a more restricted scope--these can only be accessed from within the function. Note that unlike with functions, this does not apply to the other control flow structures we have seen; for-loops and if-blocks do not have a local scope.

Let's see an example of local scoping in a function:

In [37]:
```
def print_hi():
    message = "hi"
    print(message)

print_hi()

# This generates a NameError because the variable `message` does not exist i
# it can only be accessed from inside the function.
print(message)
```

```
hi
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[37], line 9
      5 print_hi()
      7 # This generates a NameError because the variable `message` does not
exist in this scope;
      8 # it can only be accessed from inside the function.
----> 9 print(message)

NameError: name 'message' is not defined
```

The above example generates a `NameError` because the variable `message` only exists within the scope of the `print_hi()` function. By the time we try to print it again, `message` is no longer available.

> The reverse of this example is when we try to access a global variable from within a function:

In [38]:
```
message = "hi"

def print_hi():
    print(message)

print_hi()

message = "goodbye"

print_hi()
```

```
hi
goodbye
```

> In this case, we **are** able to access the variable `message` from within the function, because the variable is defined in the global scope. However, this practice is generally discouraged because it can make your code more difficult to maintain and debug.

Exercises 2

Exercise 2.8: Write a function to 'flatten' a nested list. Can you do this with a list comprehension? (Caution: The answer for a list comprehension is very simple but can be quite tricky to work out.)

`[['a','b'],['c','d']] -> ['a','b','c','d']`

In [39]:
```
X = [['a','b'],['c','d']]

flat_list = []
for sub_list in X:
    for item in sub_list:
        flat_list.append(item)
print(flat_list)

# list comprehension version
print([item for sublist in X for item in sublist])
```

```
['a', 'b', 'c', 'd']
['a', 'b', 'c', 'd']
```

Exercise 2.9: Consider the function in the following code block. Does this function work as expected? What happens to the list passed in as `my_list` when you run it? Experiment with a list of numbers.

```
def square_ith(i,my_list):
    '''Return the square of the ith element of a list'''
    x = my_list.pop(i)
    return x*x

X = [1,2,3,4,5]
print square_ith(3,X)  # Should output 16
```

In [40]:
```
def square_ith(i,my_list):
    '''Return the square of the ith element of a list'''
    x = my_list.pop(i)
    return x*x

X = [1,2,3,4,5]
print(square_ith(3,X))  # Should output 16
print(X) # uh oh! where did 4 go?
```

```
16
[1, 2, 3, 5]
```

Exercise 2.10: Write a function to create a string consisting of a single character repeated `100` times.

In [41]:
```
def repeat_string(char):
    return [char] * 100
```

Exercise 2.11: Write a function to transpose a dictionary (swap keys and values).

In [42]:
```
def transpose_dict(obj):
    return { value:key for key,value in obj.items() }
```

Exercise 2.12: Suppose you are given a list of dictionaries, each with the same keys. Write a function that converts this to a dictionary of lists.

```
    [{'a':0, 'b':1},{'a':4, 'b':7}] ->  {'a':[0, 4], 'b':[1,
7]}
```

In [43]:
```
list_of_dicts = [{'a':0, 'b':1}, {'a':4, 'b':7}]

def dict_of_lists(list_of_dicts):
    obj = {}

    for d in list_of_dicts:
```

```
        for k,v in d.items():
            if k not in obj:
                obj[k] = [v]
            else:
                obj[k].append(v)
    return obj

print(dict_of_lists(list_of_dicts))

# using defaultdict
from collections import defaultdict
def dict_of_lists_defaultdict(list_of_dicts):
    # initial default value to the empty list
    obj = defaultdict(list)

    for d in list_of_dicts:
        for k,v in d.items():
            obj[k].append(v)

    return dict(obj)

print(dict_of_lists_defaultdict(list_of_dicts))
```

```
{'a': [0, 4], 'b': [1, 7]}
{'a': [0, 4], 'b': [1, 7]}
```

Exercise 2.13: Write a function to sort the words in a string by the number of characters in each word. (Hint: `list`s have a method called `sort` that takes an optional argument called `key`. `key` allows you to specify how elements are compared for the sorting process. Try passing the function `len` as a `key` argument to `sort`.)

In [44]:
```
strings = [ "1", "4444", "333", "22" ]
strings.sort(key=len)
print (strings)
```

```
['1', '22', '333', '4444']
```

Exercise 2.14: Write a function to determine whether an integer is prime. (Hint: a number X is prime if it has no divisors other than 1 or itself. This means that there will be no integers smaller than X that divide it evenly, i.e. there will be no number y < X such that X%y==0.)

```
is_prime(9) -> False
is_prime(11) -> True
```

In [45]:
```
def is_prime(n):
    for i in range(2,n):
        if n % i == 0:
```

```
            return False
    return True
```

Exercise 2.15: Write a function that returns a list of all prime numbers less than an input value X.

In [46]:
```
def all_primes(n):
    return [ i for i in range(n) if is_prime(i) ]
print(all_primes(10))
```

```
[0, 1, 2, 3, 5, 7]
```

Exercise 2.16: Write a function to generate the Fibonnaci sequence. The first two elements of this sequence are the number 1. The next elements are generated by computing the sum of the previous two elements.

```
    1,1,2,3,5,8,...
```

In [47]:
```
def fib(n, seq=[]):
    if n == 0:
        return seq

    nseq = len(seq)
    if nseq < 2:
        return fib(n-1, [1] * (nseq+1))

    return fib(n-1, seq + [seq[-2] + seq[-1]])


print(fib(10))
```

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

Exercise 2.17: The string method `find` will return the index of the beginning of the first substring that matches a pattern,e.g.

```
'hello, world'.find('lo') -> 3
```

Write a function that will return a list of the initial positions of *all* matching substrings.

```
find_all_substrings('hello, hello, world', 'lo') -> [3, 10]
```

In [48]:
```
def find_all_substrings(s, subs):
    out_indices = []
    start_index = 0
    while start_index >= 0:
        start_index = s.find(subs)
```

```
        out_indices.append(start_index)
        s = s[start_index+1:]

    for i in range(1,len(out_indices)):
        out_indices[i] += out_indices[i-1] + 1

    # the array has a duplicate from the -1 at the end
    return out_indices[:-1]

find_all_substrings('hello, hello, world', 'lo')
```

Out[48]: [3, 10]

Exercise 2.18: Write a function that will compute the sum of all numbers less than X that are divisible by either y or z (say 3 or 5).

In [49]:
```
def complicated_sum(x, y, z):
    return sum([i for i in range(x)
                if (i % y == 0) or (i % z == 0)])
print(complicated_sum(10,3,5))
```

```
23
```

Exercise 2.19: Write a function that tests whether a string is a palindrome.

```
        "0908090" ->  True
        "212555655533" -> False
        "Doc, note: I dissent. A fast never prevents a fatness.
   I diet on cod" -> True
```

In [50]:
```
def is_palindrome(s, ignore_chars=' ,.:'):
    forward = [c.lower() for c in s if c not in ignore_chars]
    reverse = forward[::-1]
    return forward == reverse

print(is_palindrome("0908090"))
print(is_palindrome("212555655533"))
print(is_palindrome("Doc, note: I dissent. A fast never prevents a fatness.
```

```
True
False
True
```